

SQL - Funkcje okna (Window functions)

Lab 1

Imiona i nazwiska: Natalia Bratek, Jakub Karczewski

Celem ćwiczenia jest przygotowanie środowiska pracy, wstępne zapoznanie się z działaniem funkcji okna (window functions) w SQL, analiza wydajności zapytań i porównanie z rozwiązaniami przy wykorzystaniu "tradycyjnych" konstrukcji SQL

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki:

...

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- MS SQL Server - wersja 2019, 2022, 2025
- PostgreSQL - wersja 15/16/17/18
- SQLite
- Narzędzia do komunikacji z bazą danych
 - SSMS - Microsoft SQL Managment Studio
 - DDataGrip lub DBeaver
- Przykładowa baza Northwind
 - W wersji dla każdego z wymienionych serwerów

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Dokumentacja/Literatura

- Kathi Kellenberger, Clayton Groom, Ed Pollack, Expert T-SQL Window Functions in SQL Server 2019, Apres 2019
- Itzik Ben-Gan, T-SQL Window Functions: For Data Analysis and Beyond, Microsoft 2020

- Kilka linków do materiałów które mogą być pomocne - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-ver16>
 - <https://www.sqlservertutorial.net/sql-server-window-functions/>
 - <https://www.sqlshack.com/use-window-functions-sql-server/>
 - <https://www.postgresql.org/docs/current/tutorial-window.html>
 - <https://www.postgresqltutorial.com/postgresql-window-function/>
 - <https://www.sqlite.org/windowfunctions.html>
 - <https://www.sqlitetutorial.net/sqlite-window-functions/>
- W razie potrzeby - opis ikonek używanych w graficznej prezentacji planu zapytania w SSMS jest tutaj:
 - <https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference>

Przygotowanie

Uruchom SSMS - Skonfiguruj połączenie z bazą Northwind na lokalnym serwerze MS SQL

Uruchom DataGrip (lub Dbeaver)

- Skonfiguruj połączenia z bazą Northwind3
 - na lokalnym serwerze MS SQL
 - na lokalnym serwerze PostgreSQL
 - z lokalną bazą SQLite

Zadanie 1 - obserwacja

Wykonaj i porównaj wyniki następujących poleceń.

```
select avg(unitprice) avgprice
from products p;

select avg(unitprice) over () as avgprice
from products p;

select categoryid, avg(unitprice) avgprice
from products p
group by categoryid

select avg(unitprice) over (partition by categoryid) as avgprice
from products p;
```

Jaka jest są podobieństwa, jakie różnice pomiędzy grupowaniem danych a działaniem funkcji okna?

Wyniki:

100 %

Results		Messages	
	avgprice		
1	28,8663		

Results		Messages	
	avgprice		
1	28,8663		
2	28,8663		
3	28,8663		
4	28,8663		
5	28,8663		
6	28,8663		
7	28,8663		
8	28,8663		
9	28,8663		
10	28,8663		
11	28,8663		

100 %

Results		Messages	
	categoryid	avgprice	
1	1	37,9791	
2	2	23,0625	
3	3	25,16	
4	4	28,73	
5	5	20,25	
6	6	54,0066	
7	7	32,37	
8	8	20,6825	

100 %

Results Messages

	avgprice
1	37,9791
2	37,9791
3	37,9791
4	37,9791
5	37,9791
6	37,9791
7	37,9791
8	37,9791
9	37,9791
10	37,9791
11	37,9791
12	37,9791
13	23,0625
14	23,0625
15	23,0625
16	23,0625
17	23,0625
18	23,0625
19	23,0625
20	23,0625
21	23,0625
22	23,0625
23	23,0625
24	23,0625

- podobieństwa: funkcja okna i grupowanie dają takie same wyniki dla średniej ceny kategorii i produktów
- różnice: grupowanie zwraca jeden wynik dla każdej grupy, a funkcja okna zwraca wszystkie rekordy razem z obliczonym wynikiem

Zadanie 2 - obserwacja

Wykonaj i porównaj wyniki następujących poleceń.

```
--1)

select p.productid, p.ProductName, p.unitprice,
       (select avg(unitprice) from products) as avgprice
from products p
where productid < 10

--2)

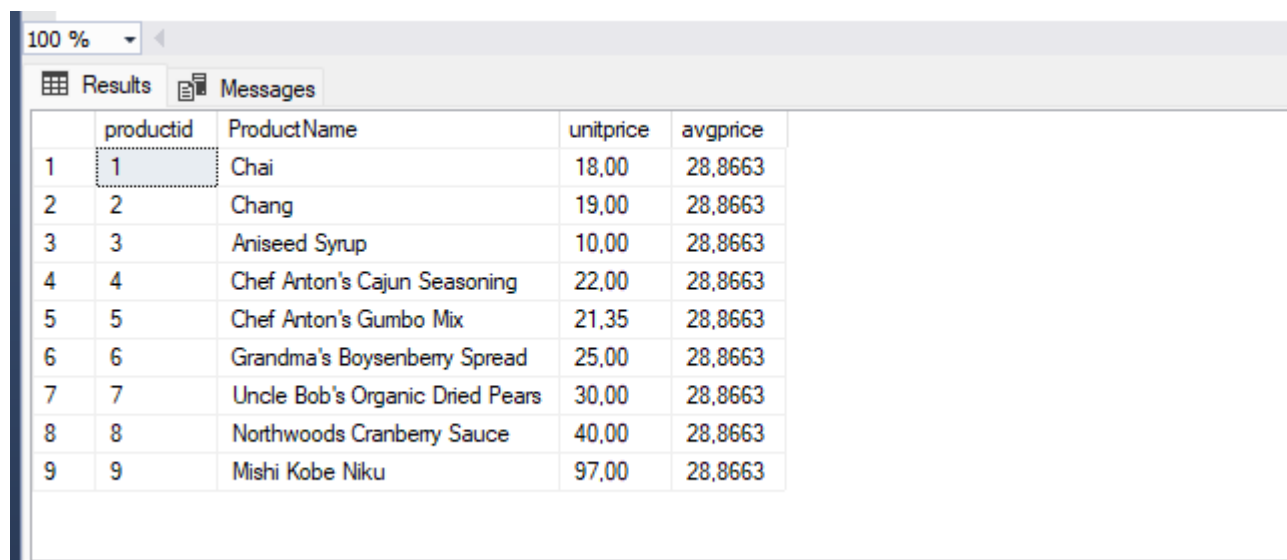
select p.productid, p.ProductName, p.unitprice,
       avg(unitprice) over () as avgprice
```

```
from products p
where productid < 10
```

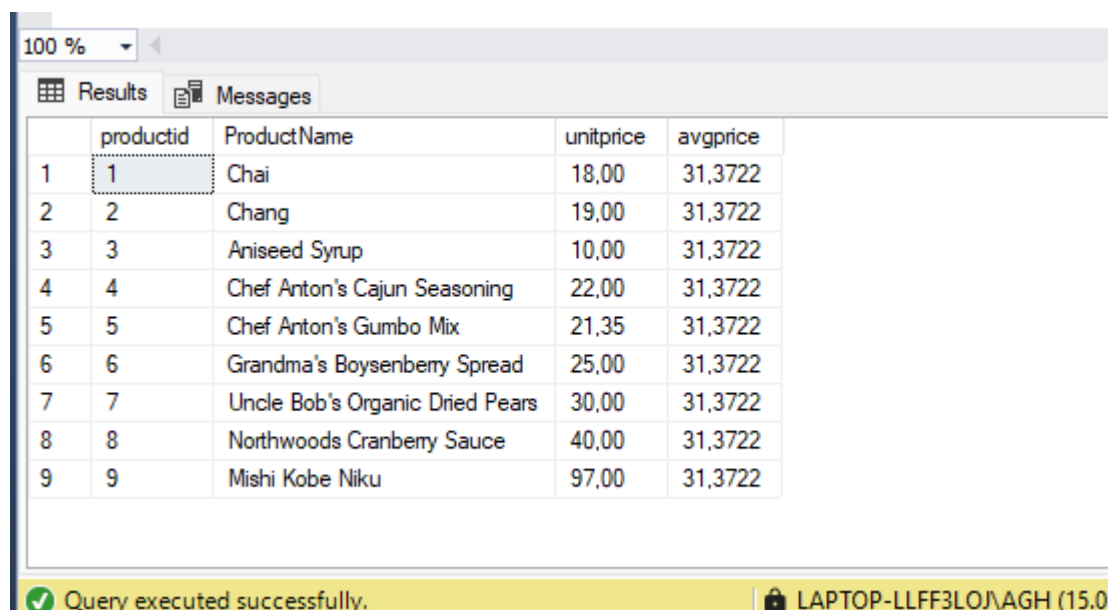
Jaka jest różnica? Czego dotyczy warunek w każdym z przypadków? Napisz polecenie równoważne

- 1. z wykorzystaniem funkcji okna. Napisz polecenie równoważne
- 2. z wykorzystaniem podzapytania

Wyniki:



	productid	ProductName	unitprice	avgprice
1	1	Chai	18,00	28,8663
2	2	Chang	19,00	28,8663
3	3	Aniseed Syrup	10,00	28,8663
4	4	Chef Anton's Cajun Seasoning	22,00	28,8663
5	5	Chef Anton's Gumbo Mix	21,35	28,8663
6	6	Grandma's Boysenberry Spread	25,00	28,8663
7	7	Uncle Bob's Organic Dried Pears	30,00	28,8663
8	8	Northwoods Cranberry Sauce	40,00	28,8663
9	9	Mishi Kobe Niku	97,00	28,8663



	productid	ProductName	unitprice	avgprice
1	1	Chai	18,00	31,3722
2	2	Chang	19,00	31,3722
3	3	Aniseed Syrup	10,00	31,3722
4	4	Chef Anton's Cajun Seasoning	22,00	31,3722
5	5	Chef Anton's Gumbo Mix	21,35	31,3722
6	6	Grandma's Boysenberry Spread	25,00	31,3722
7	7	Uncle Bob's Organic Dried Pears	30,00	31,3722
8	8	Northwoods Cranberry Sauce	40,00	31,3722
9	9	Mishi Kobe Niku	97,00	31,3722

Query executed successfully. LAPTOP-LLFF3LOJ\AGH (15.0)

Różnica: w pierwszym zapytaniu średnia liczona jest w podzapytaniu, które nie widzi warunku where productid < 10 z głównego zapytania. Dlatego avg() bierze pod uwagę wszystkie produkty z tabeli products, a potem ta wartość jest doklejana do każdego rekordu spełniającego warunek. W drugim zapytaniu występuje funkcja okna. Najpierw działa where, zostają tylko produkty o id < 10 i dopiero z nich liczona jest średnia.

Podzapytanie równoważne 2

```
SELECT
  p.productid,
  p.ProductName,
  p.unitprice,
  (SELECT AVG(unitprice)
   FROM products
   WHERE productid < 10) AS avgprice
FROM products p
WHERE productid < 10;
```

	productid	ProductName	unitprice	avgprice
1	1	Chai	18,00	31,3722
2	2	Chang	19,00	31,3722
3	3	Aniseed Syrup	10,00	31,3722
4	4	Chef Anton's Cajun Seasoning	22,00	31,3722
5	5	Chef Anton's Gumbo Mix	21,35	31,3722
6	6	Grandma's Boysenberry Spread	25,00	31,3722
7	7	Uncle Bob's Organic Dried Pears	30,00	31,3722
8	8	Northwoods Cranberry Sauce	40,00	31,3722
9	9	Mishi Kobe Niku	97,00	31,3722

funkcja okna równoważna 1

```
WITH calc AS (
  SELECT *, AVG(unitprice) OVER () AS avgprice
  FROM products
)
SELECT productid, ProductName, unitprice, avgprice
FROM calc
WHERE productid < 10;
```

Results		Messages		
	productid	ProductName	unitprice	avgprice
1	1	Chai	18,00	28,8663
2	2	Chang	19,00	28,8663
3	3	Aniseed Syrup	10,00	28,8663
4	4	Chef Anton's Cajun Seasoning	22,00	28,8663
5	5	Chef Anton's Gumbo Mix	21,35	28,8663
6	6	Grandma's Boysenberry Spread	25,00	28,8663
7	7	Uncle Bob's Organic Dried Pears	30,00	28,8663
8	8	Northwoods Cranberry Sauce	40,00	28,8663
9	9	Mishi Kobe Niku	97,00	28,8663

Zadanie 3

Baza: Northwind, tabela: products

Napisz polecenie, które zwraca: id produktu, nazwę produktu, cenę produktu, średnią cenę wszystkich produktów.

Napisz polecenie z wykorzystaniem z wykorzystaniem podzapytania, join'a oraz funkcji okna. Porównaj czasy oraz plany wykonania zapytań.

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

W SSMS włącz dwie opcje: Include Actual Execution Plan oraz Include Live Query Statistics

The screenshot shows the SQL Server Enterprise Manager interface. At the top, a query window displays the following SQL code:

```
1 select p.productid, p.ProductName, p.unitprice,  
2       (select avg(unitprice) from products) as avgprice  
3 from products p;
```

Below the query window, the 'Execution plan' tab is selected. It shows the query cost (relative to the batch) as 100%. The execution plan diagram illustrates the following steps:

- SELECT**: Cost: 0 %
- Compute Scalar**: Cost: 0 %
- Nested Loops (Inner Join)**: Cost: 5 %, 0.000s, 77 of 77 (100%)
- Compute Scalar**: Cost: 0 %
- Stream Aggregate (Aggregate)**: Cost: 1 %, 0.000s, 1 of 1 (100%)
- Clustered Index Scan (Clustered) [Products].[PK_Products]**: Cost: 47 %, 0.000s, 77 of 77 (100%)
- Clustered Index Scan (Clustered) [Products].[PK_Products] [p]**: Cost: 47 %, 0.000s, 77 of 77 (100%)

W DataGrip użyj opcji Explain Plan/Explain Analyze

The screenshot shows the DataGrip interface. At the top, a query window displays the following SQL code:

```
1 select p.productid, p.ProductName, p.unitprice,  
2       (select avg(unitprice) from products) as avgprice  
3 from products p;
```

Below the query window, the 'Execution plan' tab is selected. A context menu is open, showing the following options:

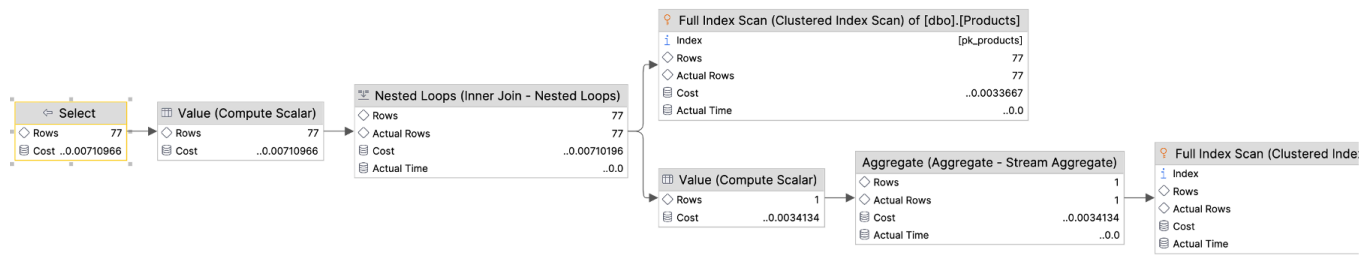
- Show Context Actions
- Paste
- Copy / Paste Special
- Column Selection Mode
- Refactor
- Folding
- Save as Live Template...
- Reformat Code
- Go To
- Generate...
- Run '.l.sql'
- More Run/Debug
- Switch Session (w)
- Explain Plan**
- Execute
- Execute to File
- Open In

The 'Explain Plan' option is highlighted, and a sub-menu is open, showing the following options:

- Explain Plan
- Explain Plan (Raw)
- Explain Analyze**
- Explain Analyze (Raw)

Below the context menu, the 'Services' panel is visible, showing the 'Operation' tree. The 'Explain Plan' operation is selected, and the 'Explain Analyze' option is highlighted. The 'Explain Analyze' operation is also selected in the 'Services' panel.

	Rows	Actual Rows	Total Cost	Actual Total Ti
	77		0.00710966	
	77		0.00710966	
	77	77	0.00710196	0.0
	1		0.0034134	
	1	1	0.0034134	0.0
[PK_Products];	77	77	0.0033667	0.0
[PK_Products];	77	77	0.0033667	0.0



Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time
Select		77		0.00710966	
Value (Compute Scalar)		77		0.00710966	
Value (Compute Scalar)	(Inner Join - N	77	77	0.00710196	0.0
Value (Compute Scalar)		1		0.0034134	
Aggregate (Aggregate - S		1	1	0.0034134	0.0
Full Index Scan (Clu	table: [dbo].[Products]; index: [PK_Products];	77	77	0.0033667	0.0
Full Index Scan (Clustered	table: [dbo].[Products]; index: [PK_Products];	77	77	0.0033667	0.0

Wyniki:

- podzapytanie

```
select productid, productname, unitprice,
(select avg(unitprice) from products) as avgprice
from products p;
```

- funkcja okna

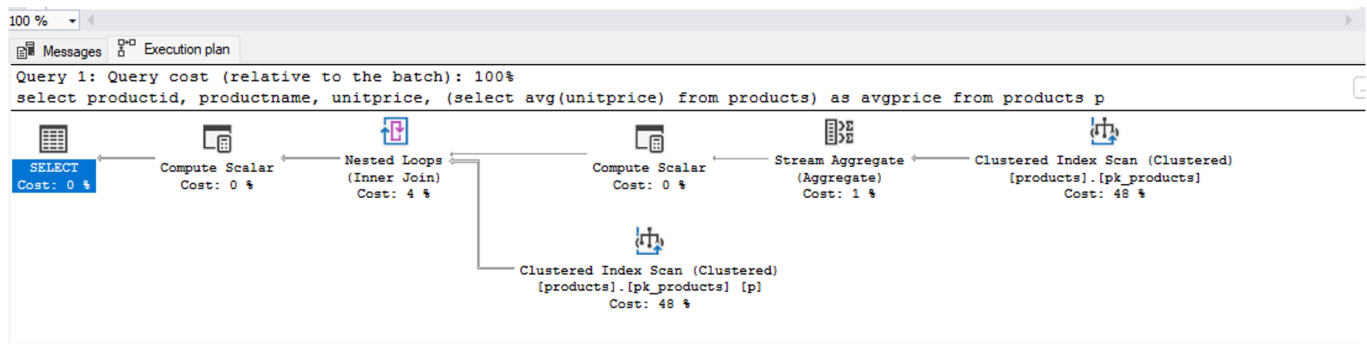
```
select productid, productname, unitprice,
avg(unitprice) over () as avgprice
from products;
```

- join

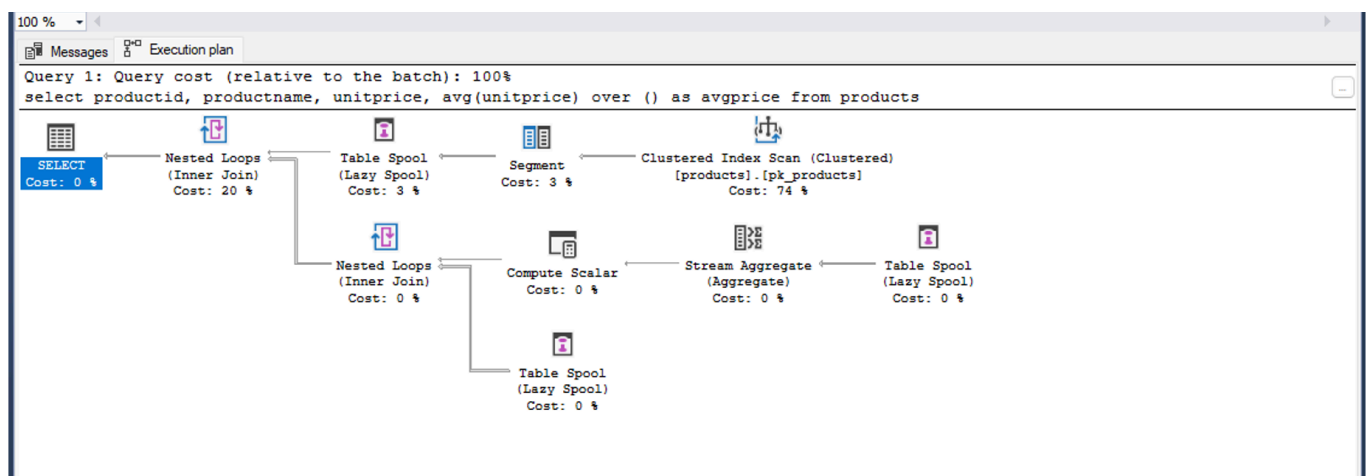
```
select p.productid, p.productname, p.unitprice, avg(a.unitprice) as
avgprice
from products p
cross join products a
group by p.productid, p.productname, p.unitprice;
```

Plany zapytań dla MS SQL Server

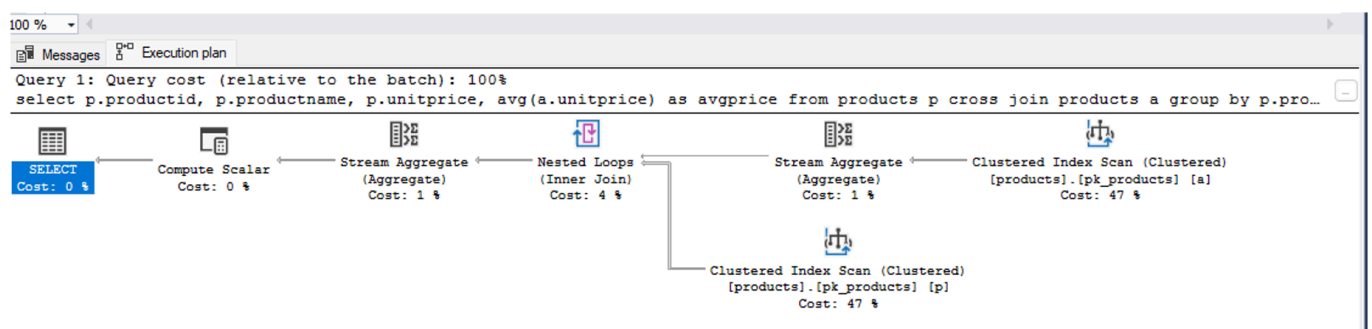
- podzapytanie



- funkcja okna

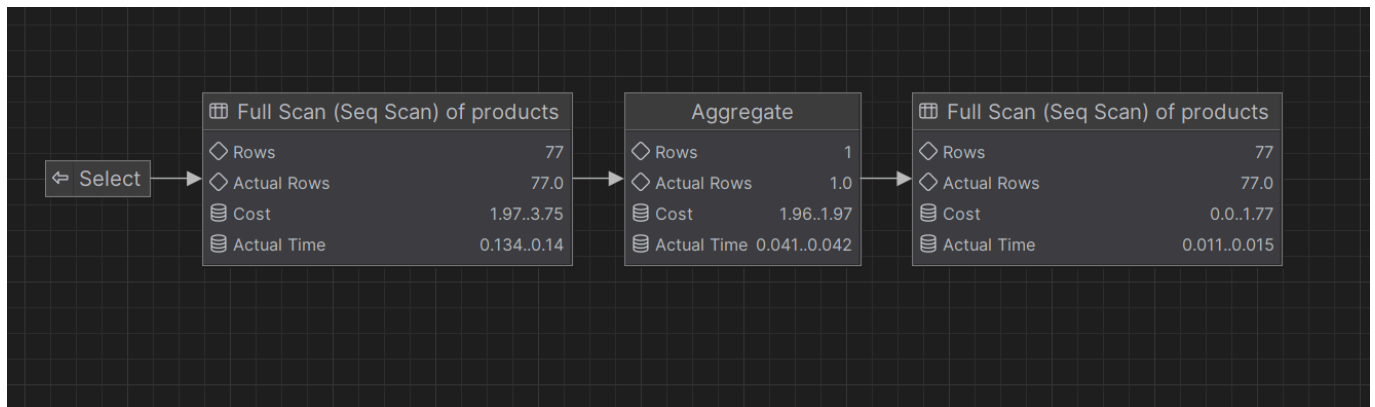


- join

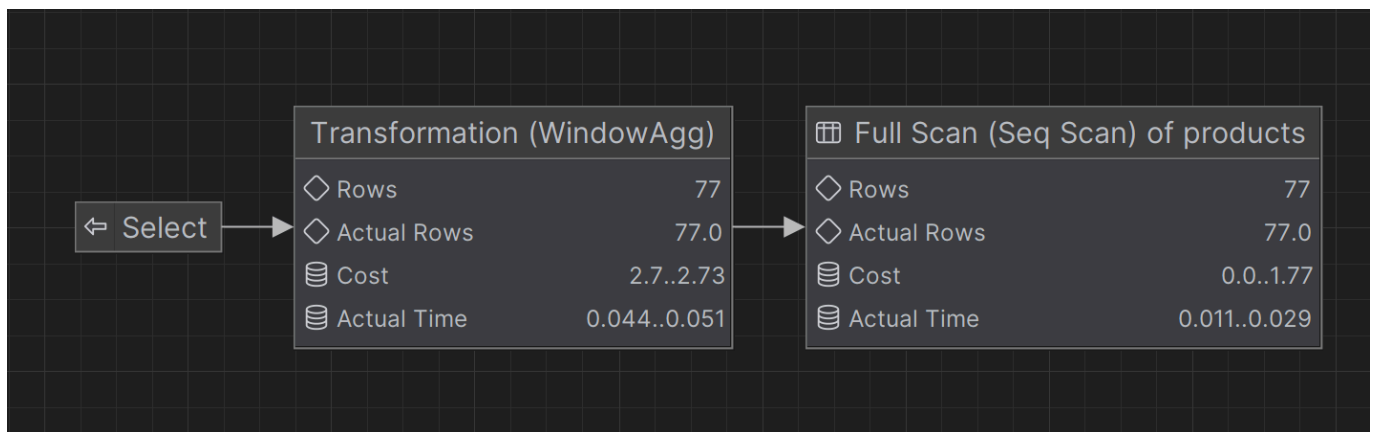


Plany zapytań dla PostgreSQL

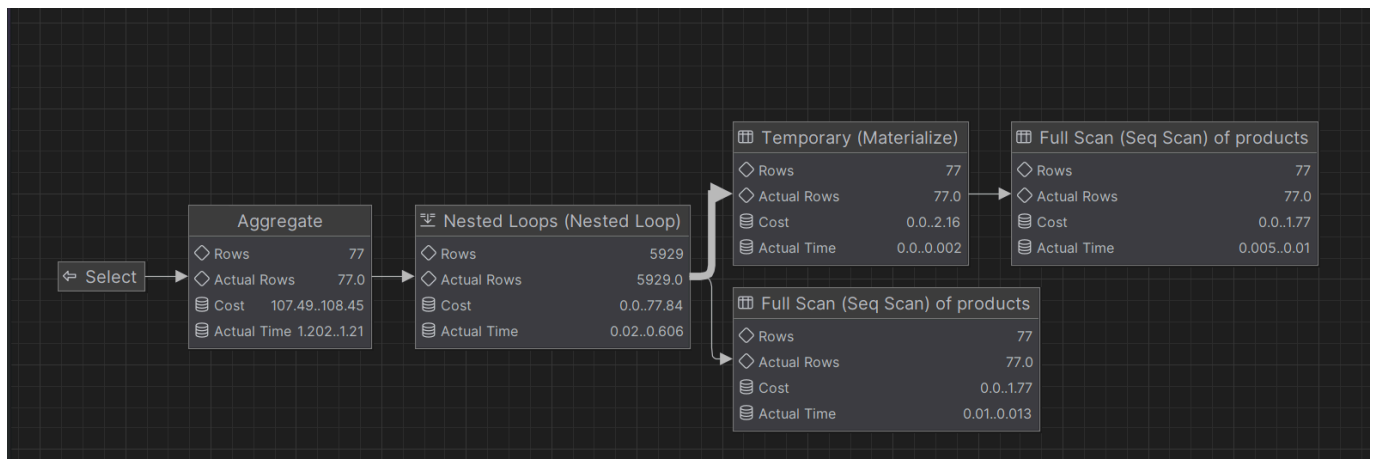
- podzapytanie



- funkcja okna

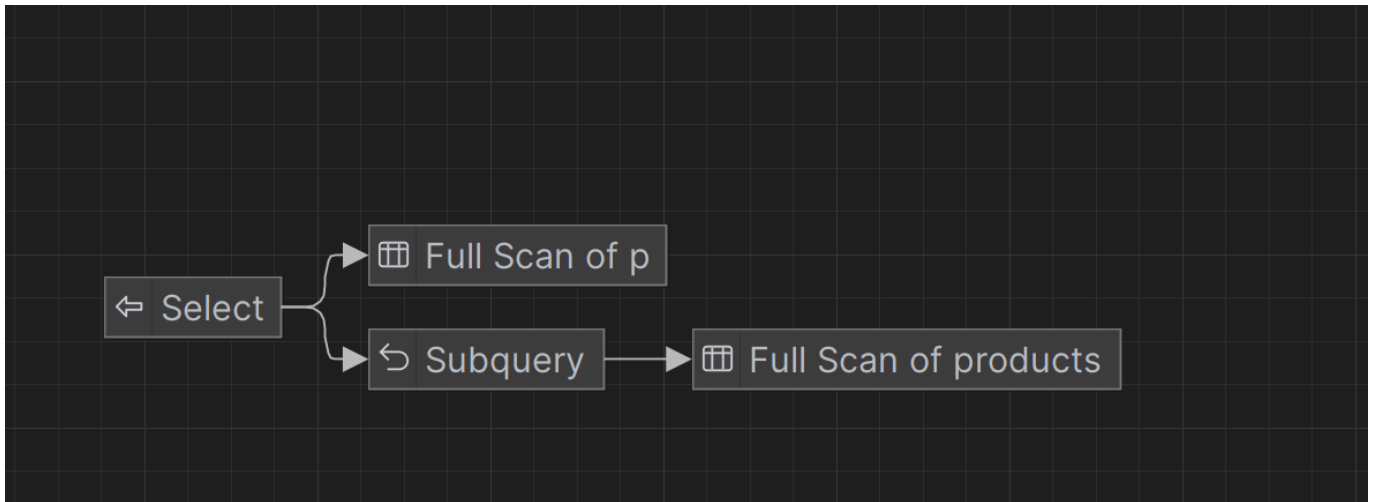


- join

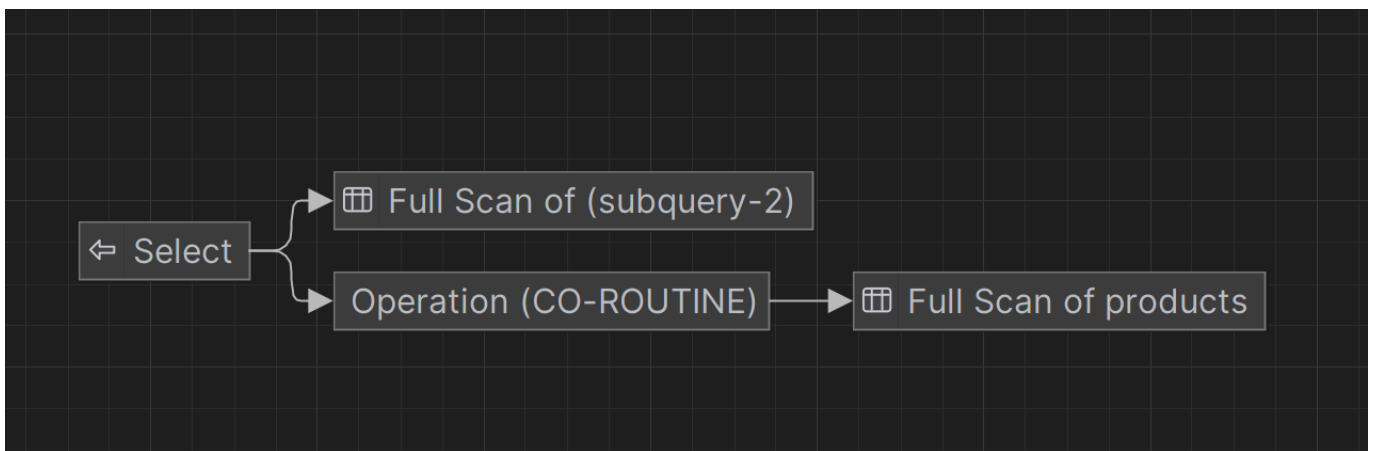


Plany zapytań dla SQLite

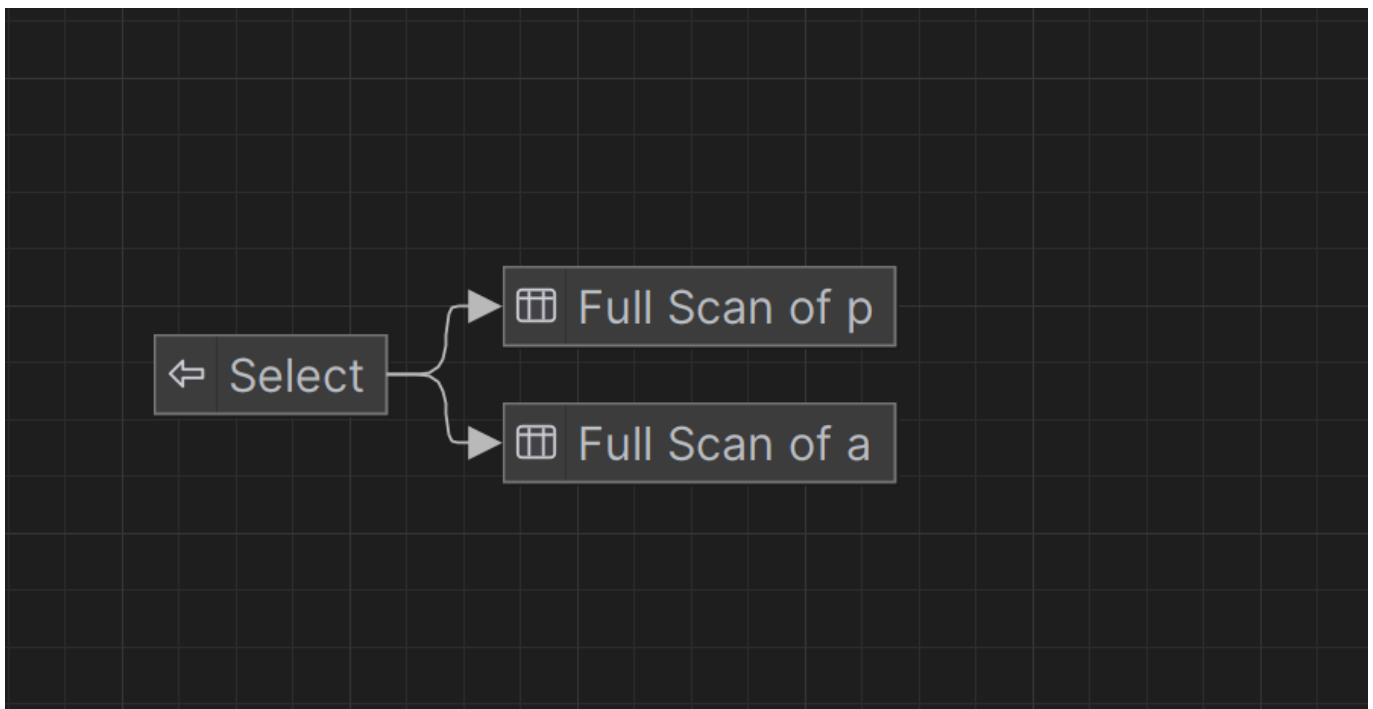
- podzapytanie



- funkcja okna



- join



Porównanie

MS SQL SERVER

- Podzapytanie i join mają podobne plany. Oba wykonują dwa skanowania tabeli products
- Funkcja okna ma jeden skan tabeli, ale ma więcej operacji

PostgreSQL

- Podzapytanie ma koszt 3.75, występują dwa Seq Scan dla tabeli products
- Funkcja okna ma koszt 2.73, jest tylko jeden Seq Scan i WindowAgg
- Join ma koszt 108, bo cross join tworzy 5929 wierszy ($77 * 77$)
- Najlepszy koszt ma funkcja okna, a najgorszy join

SQLite

- Wszystkie trzy plany mają Full Scan tabeli products dwa razy
- Plany wyglądają podobnie

Zadanie 4

Baza: Northwind, tabela products

Napisz polecenie, które zwraca: id produktu, nazwę produktu, cenę produktu, średnią cenę produktów w kategorii, do której należy dany produkt. Wyświetl tylko pozycje (produkty) których cena jest większa niż średnia cena.

Napisz polecenie z wykorzystaniem podzapytania, join'a oraz funkcji okna. Porównaj zapytania. Porównaj czasy oraz plany wykonania zapytań.

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

- podzapytanie

```
select p.productid, p.productname, p.unitprice,
       (select avg(x.unitprice)
        from products x
        where x.categoryid = p.categoryid
       ) as avg_category_price
from products p
where p.unitprice > (
    select avg(x.unitprice)
    from products x
    where x.categoryid = p.categoryid
);
```

- funkcja okna

```

select *
from (
    select p.productid, p.productname, p.unitprice,
           avg(p.unitprice) over (partition by p.categoryid) as
avg_category_price
    from products p
) t
where t.unitprice > t.avg_category_price;

```

- join

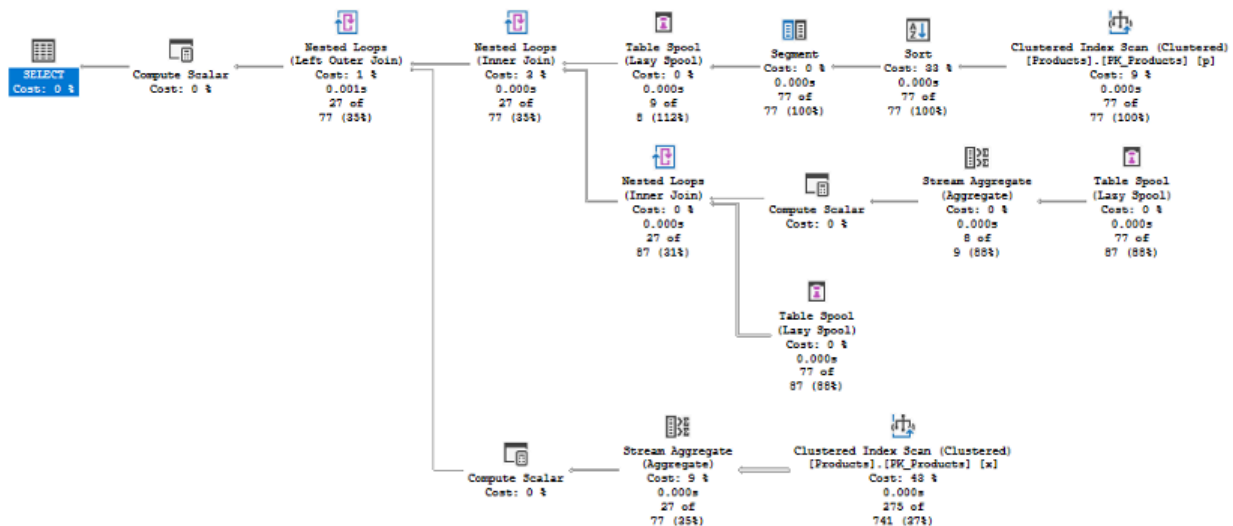
```

select p.productid, p.productname, p.unitprice, avg(x.unitprice) as
avg_category_price
from products p
left join products x
    on x.categoryid = p.categoryid
group by p.productid, p.productname, p.unitprice
having p.unitprice > avg(x.unitprice);

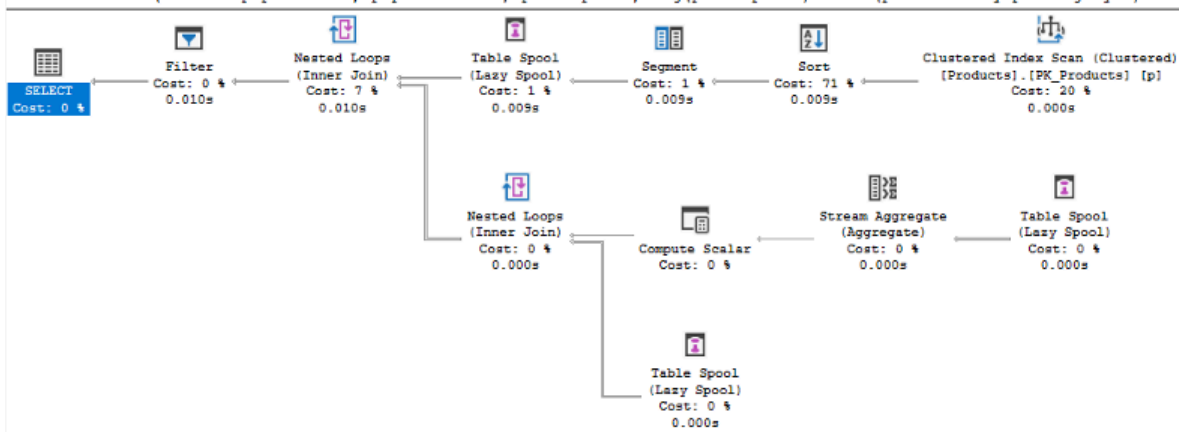
```

Plany zapytań dla MS SQL Server

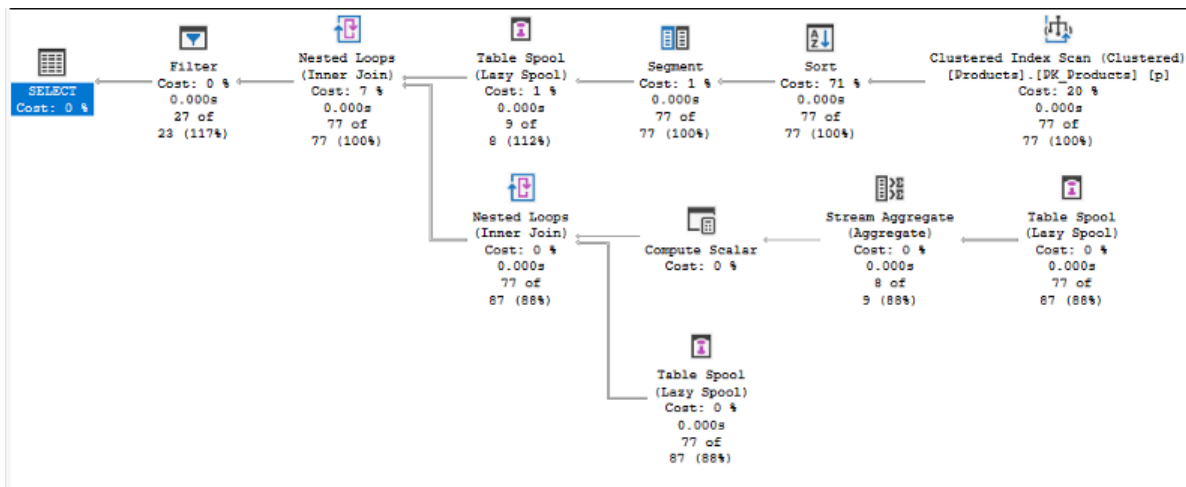
- podzapytanie



- funkcja okna

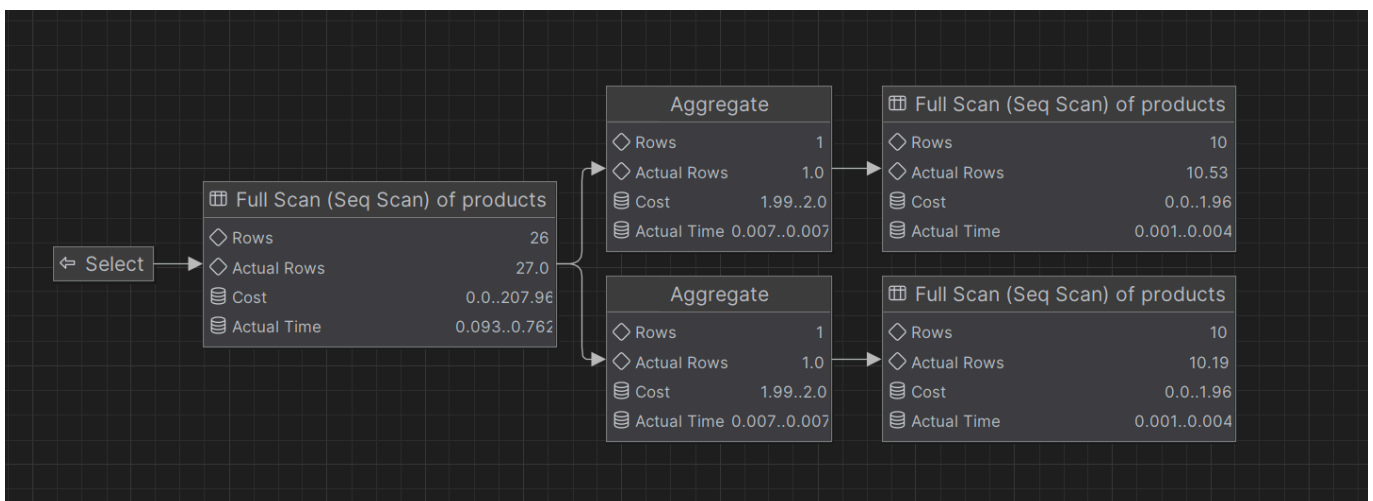


- join

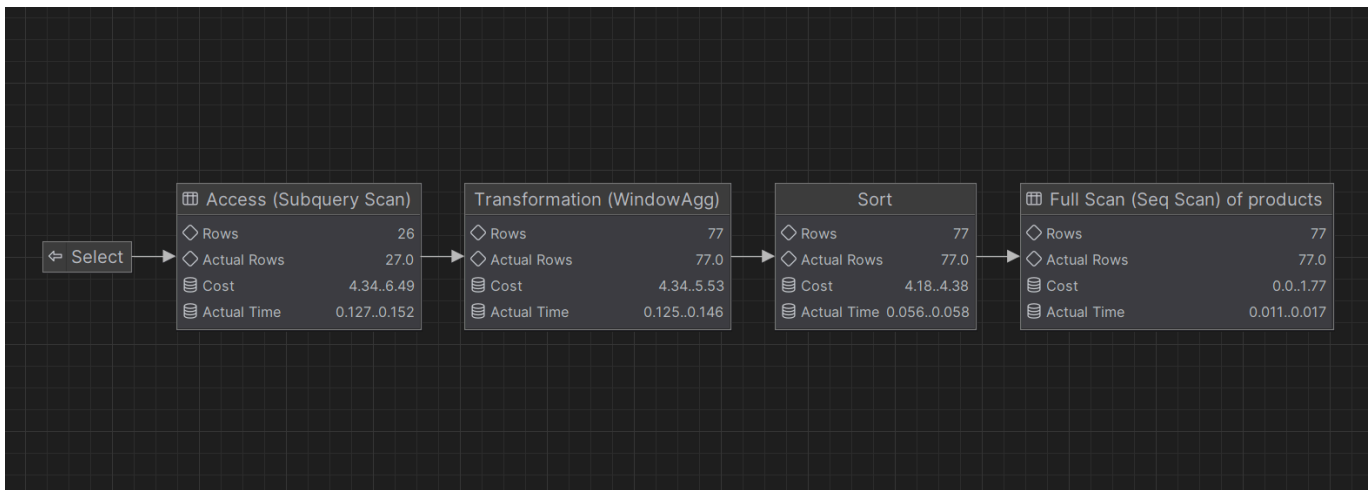


Plany zapytań dla PostgreSQL

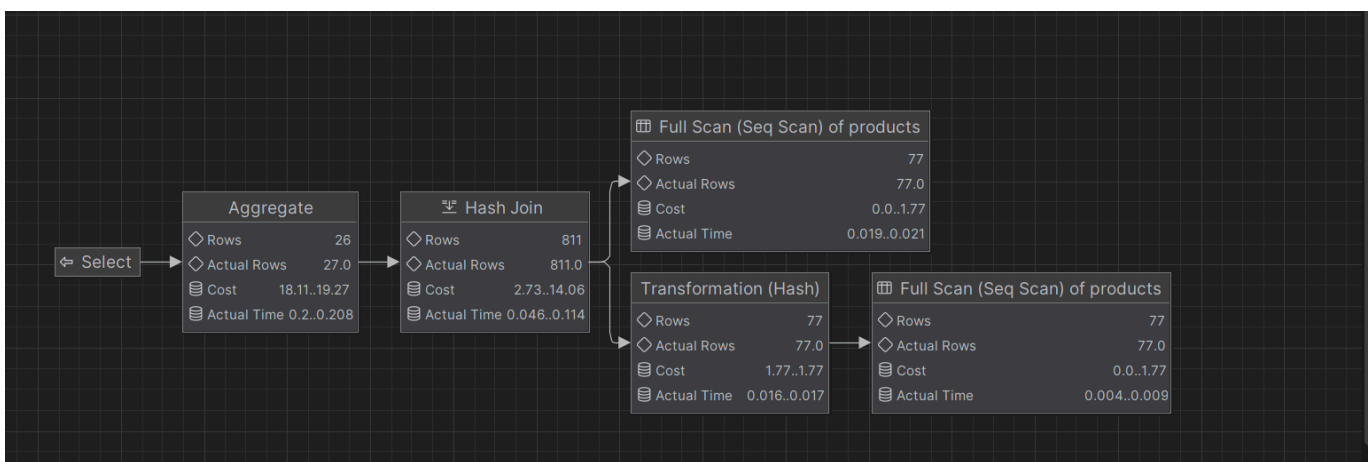
- podzapytanie



- funkcja okna

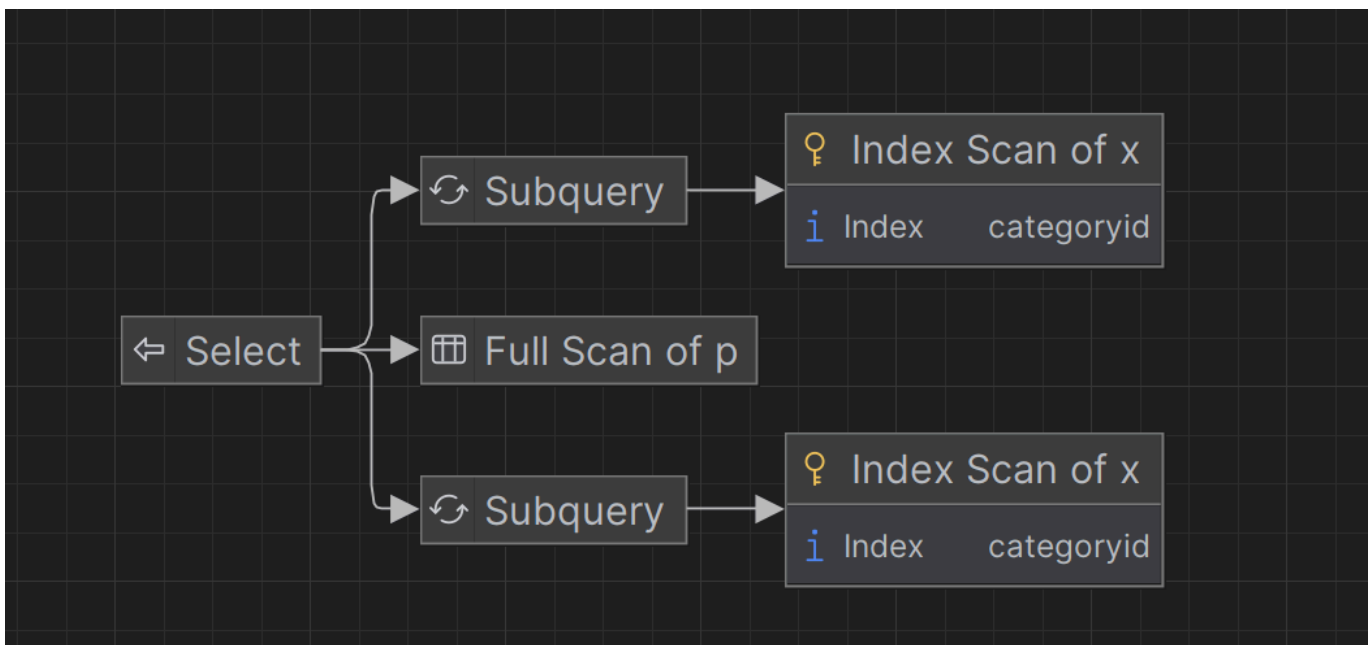


- join

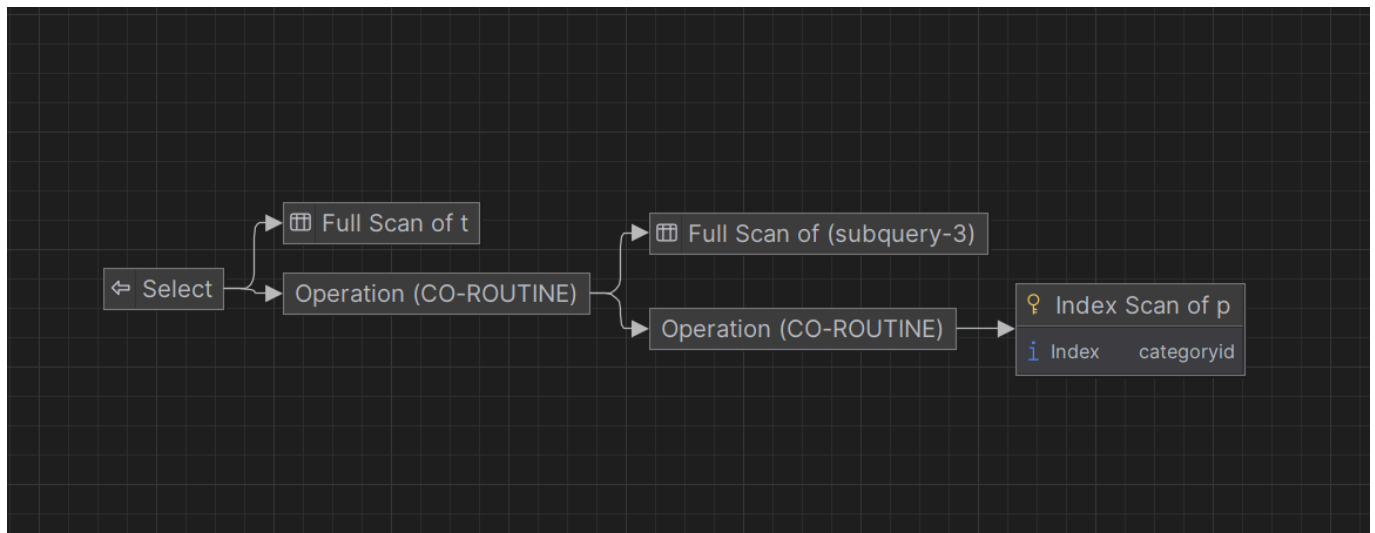


Plany zapytań dla SQLite

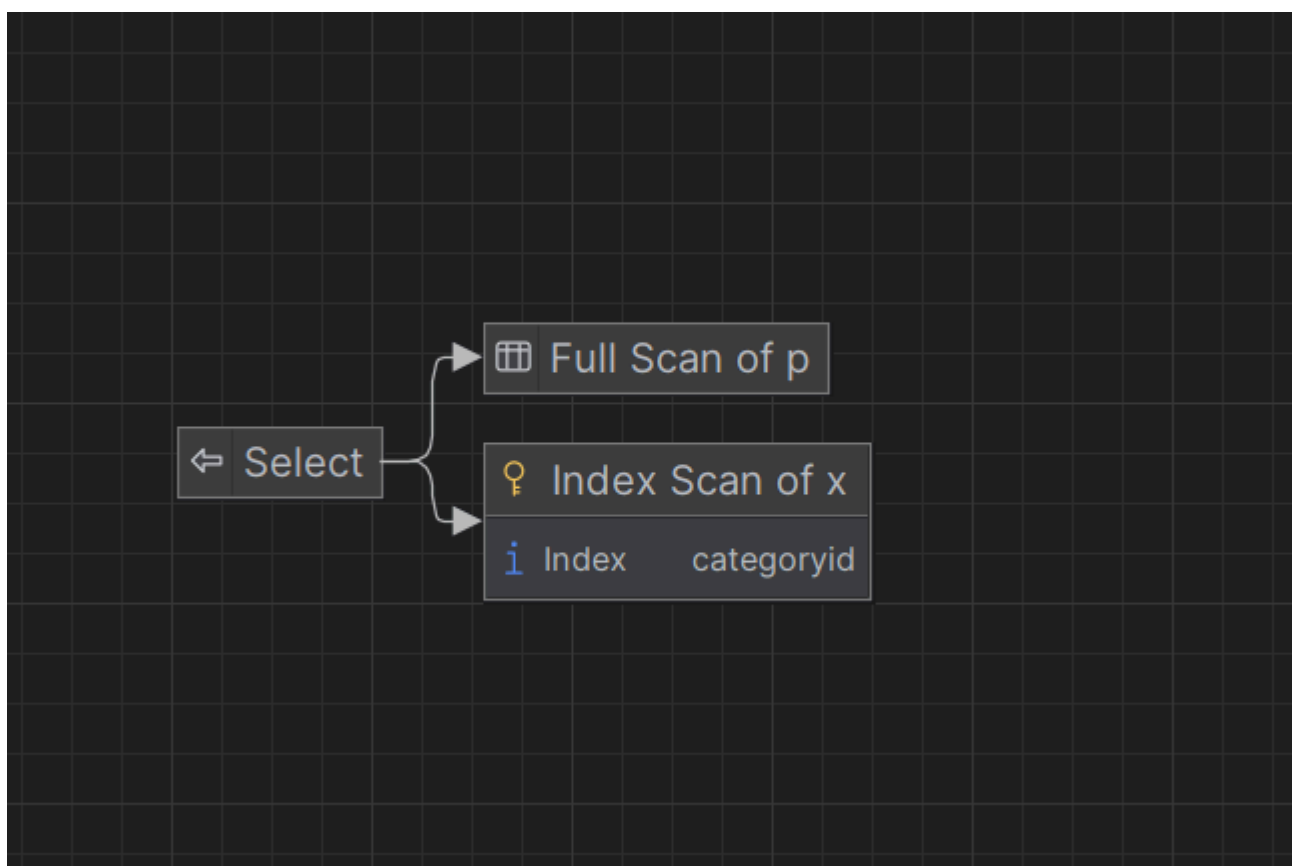
- podzapytanie



- funkcja okna



- join



Wnioski

MS SQL Server

- Podzapytanie ma dwa Clustered Index Scan dla tabeli products
- Funkcja okna ma jeden Clustered Index Scan i mniej operacji niż podzapytanie
- Join ma jeden Clustered Index Scan, plan podobny do funkcji okna

PostgreSQL

- Podzapytanie ma koszt 207.96, występuje Seq Scan tabeli products trzy razy, ma najwyższy koszt w porównaniu do pozostałych zapytań

- Funkcja okna ma koszt 6.49, jest jeden Seq Scan, WindowAgg i Sort
- Join ma koszt 19.27, używa Hash Join
- Najlepszy koszt ma funkcja okna, a najgorszy podzapytanie

SQLite

- Podzapytanie używa dwa razy Index Scan po categoryid, występuje jeden Full Scan
- Funkcja okna ma najwięcej operacji
- Join ma najprostsz plan: Full Scan of p i Index Scan of x

Zadanie 5

Oryginalna baza Northwind jest bardzo mała. Warto zaobserwować działanie na nieco większym zbiorze danych.

Baza Northwind3 zawiera dodatkową tabelę product_history

- 2,2 mln wierszy

Bazę Northwind3 można pobrać z moodle (zakładka - Backupy baz danych)

Można też wygenerować tabelę product_history przy pomocy skryptu

Skrypt dla SQL Sriver

Stwórz tabelę o następującej strukturze:

```
create table product_history(  
  id int identity(1,1) not null,  
  productid int,  
  productname varchar(40) not null,  
  supplierid int null,  
  categoryid int null,  
  quantityperunit varchar(20) null,  
  unitprice decimal(10,2) null,  
  quantity int,  
  value decimal(10,2),  
  date date,  
  constraint pk_product_history primary key clustered  
    (id asc )  
)
```

Wygeneruj przykładowe dane:

Dla 30000 iteracji, tabela będzie zawierała nieco ponad 2mln wierszy (dostostu ograniczenie do możliwości swojego komputera)

Skrypt dla SQL Sriver

```

declare @i int
set @i = 1
while @i <= 30000
begin
    insert product_history
    select productid, ProductName, SupplierID, CategoryID,
        QuantityPerUnit, round(RAND()*unitprice + 10,2),
        cast(RAND() * productid + 10 as int), 0,
        dateadd(day, @i, '1940-01-01')
    from products
    set @i = @i + 1;
end;

update product_history
set value = unitprice * quantity
where 1=1;

```

Skrypt dla Postgresql

```

create table product_history(
    id int generated always as identity not null
        constraint pkproduct_history
            primary key,
    productid int,
    productname varchar(40) not null,
    supplierid int null,
    categoryid int null,
    quantityperunit varchar(20) null,
    unitprice decimal(10,2) null,
    quantity int,
    value decimal(10,2),
    date date
);

```

Wygeneruj przykładowe dane:

Skrypt dla Postgresql

```

do $$
begin
    for cnt in 1..30000 loop
        insert into product_history(productid, productname, supplierid,
            categoryid, quantityperunit,
            unitprice, quantity, value, date)
        select productid, productname, supplierid, categoryid,
            quantityperunit,
            round((random()*unitprice + 10)::numeric,2),
            cast(random() * productid + 10 as int), 0,

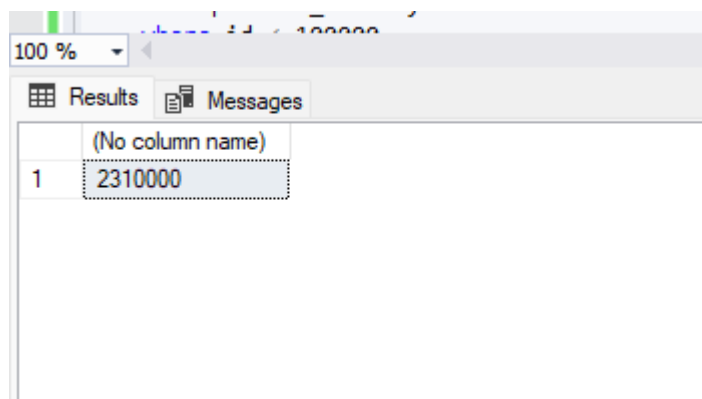
```

```
        cast('1940-01-01' as date) + cnt
    from products;
end loop;
end; $$;

update product_history
set value = unitprice * quantity
where 1=1;
```

Wykonaj polecenia: `select count(*) from product_history`, potwierdzające wykonanie zadania

Wyniki:



The screenshot shows a SQL Server interface with a 'Results' tab selected. The results pane displays a single row with two columns. The first column is labeled '(No column name)' and contains the value '1'. The second column contains the value '2310000'. The interface also shows a zoom level of 100% and tabs for 'Results' and 'Messages'.

(No column name)	
1	2310000

Zadanie 6

Baza: Northwind, tabela product_history

Napisz polecenie, które zwraca: id pozycji, id produktu, nazwę produktu, id_kategorii, cenę produktu, średnią cenę produktów w kategorii do której należy dany produkt. Wyświetl tylko pozycje (produkty) których cena jest większa niż średnia cena.

W przypadku długiego czasu wykonania ogranicz zbiór wynikowy do kilkuset/kilku tysięcy wierszy

pomocna może być konstrukcja `with`

```
with t as (
    ....
)
select * from t
where id between ....
```

Napisz polecenie z wykorzystaniem podzapytania, join'a oraz funkcji okna. Porównaj zapytania. Porównaj czasy oraz plany wykonania zapytań.

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

- podzapytanie

```
WITH t AS (  
    SELECT p.id, p.productid, p.productname, p.categoryid, p.unitprice,  
           (SELECT AVG(p2.unitprice)  
            FROM product_history p2  
            WHERE p2.categoryid = p.categoryid  
           ) AS avg_category_price  
    FROM product_history p  
    WHERE p.id BETWEEN 1 AND 1999  
)  
SELECT *  
FROM t  
WHERE unitprice > avg_category_price  
ORDER BY id;
```

- join

```
WITH t AS (  
    SELECT *  
    FROM product_history  
    WHERE id < 2000  
)  
SELECT p.id, p.productid, p.productname, p.categoryid, p.unitprice,  
       AVG(pp.unitprice) AS avg_price_in_category  
FROM t p  
JOIN t pp ON p.categoryid = pp.categoryid  
GROUP BY p.id, p.productid, p.productname, p.categoryid, p.unitprice  
HAVING p.unitprice > AVG(pp.unitprice)  
ORDER BY p.id;
```

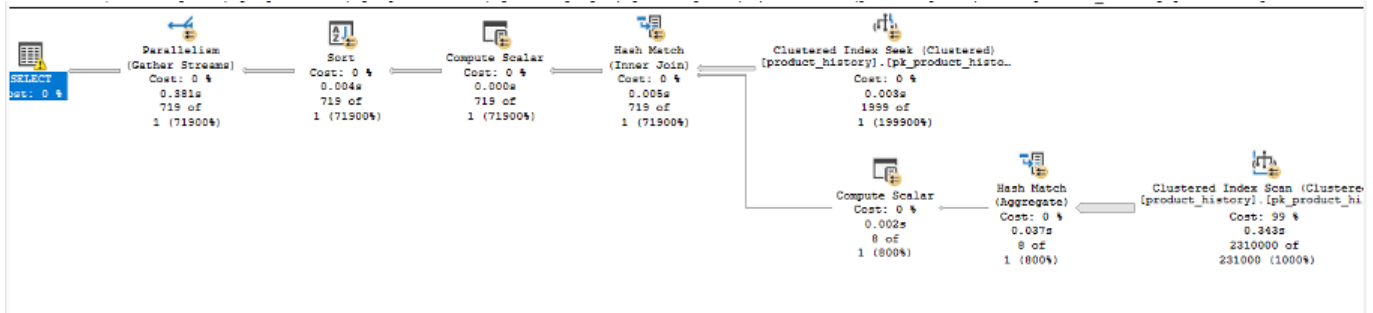
- funkcja okna

```
WITH t AS (  
    SELECT *  
    FROM product_history  
    WHERE id < 2000  
)  
SELECT *  
FROM (  
    SELECT id, productid, productname, categoryid, unitprice,  
           AVG(unitprice) OVER (PARTITION BY categoryid) AS  
avg_price_in_category  
    FROM t  
) x
```

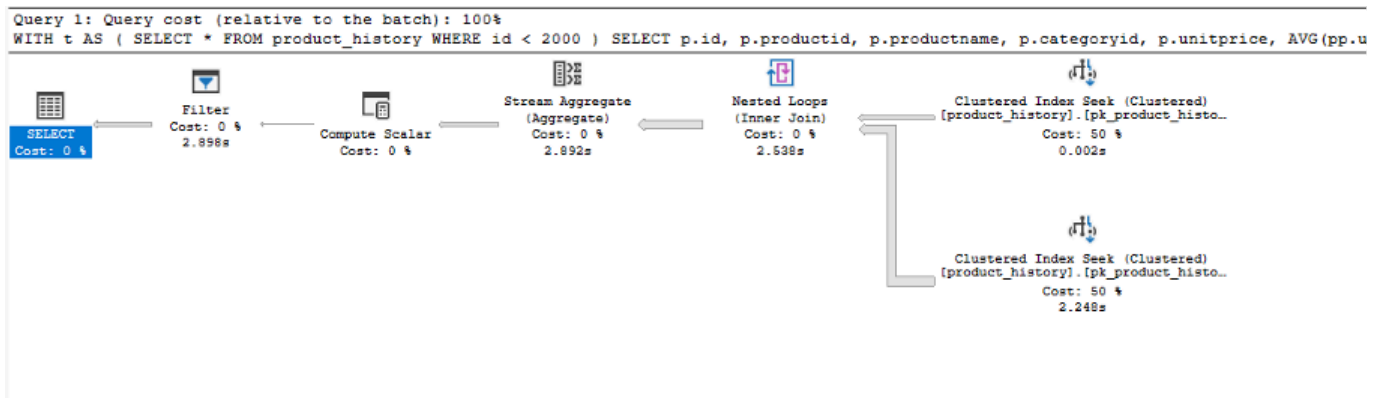
```
WHERE unitprice > avg_price_in_category
ORDER BY id;
```

Wyniki zapytań dla MS SQL Server

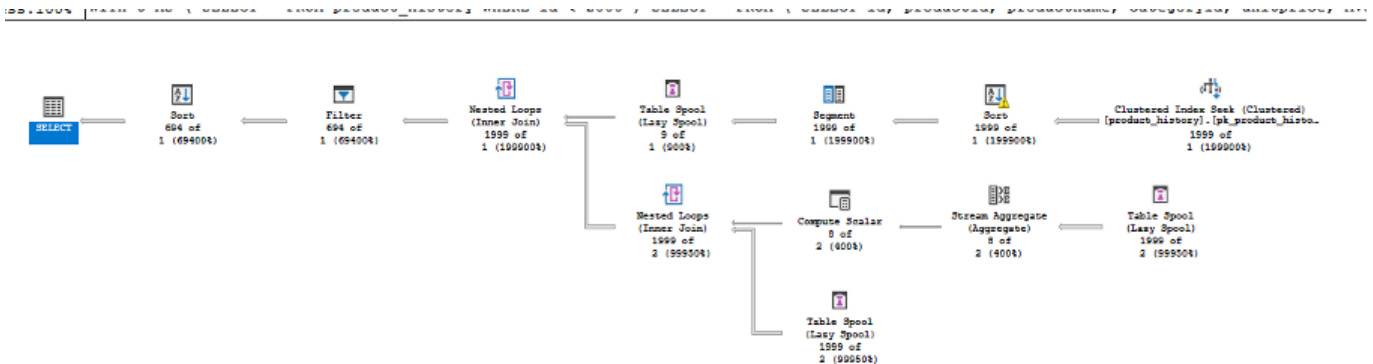
- podzapytanie



- join

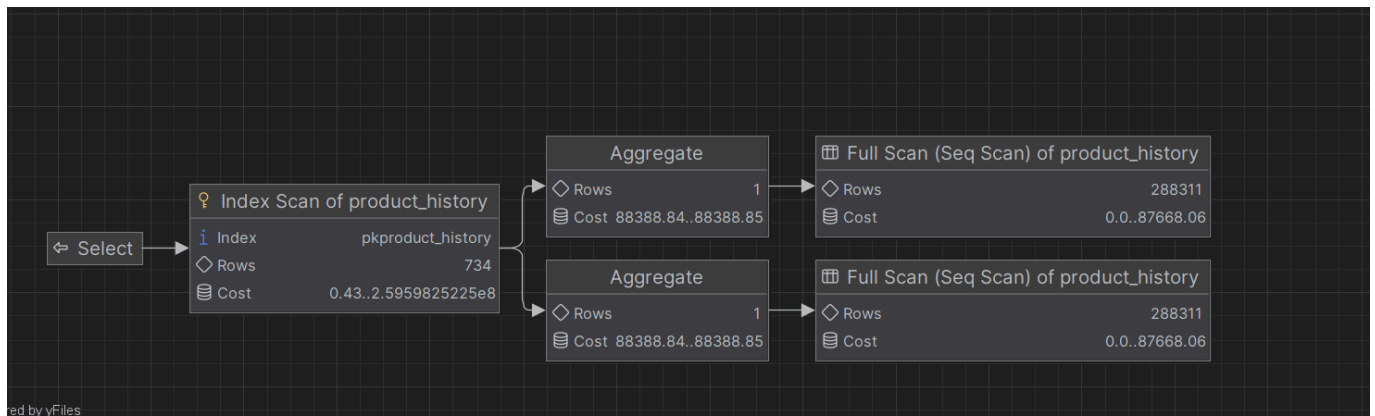


- funkcja okna

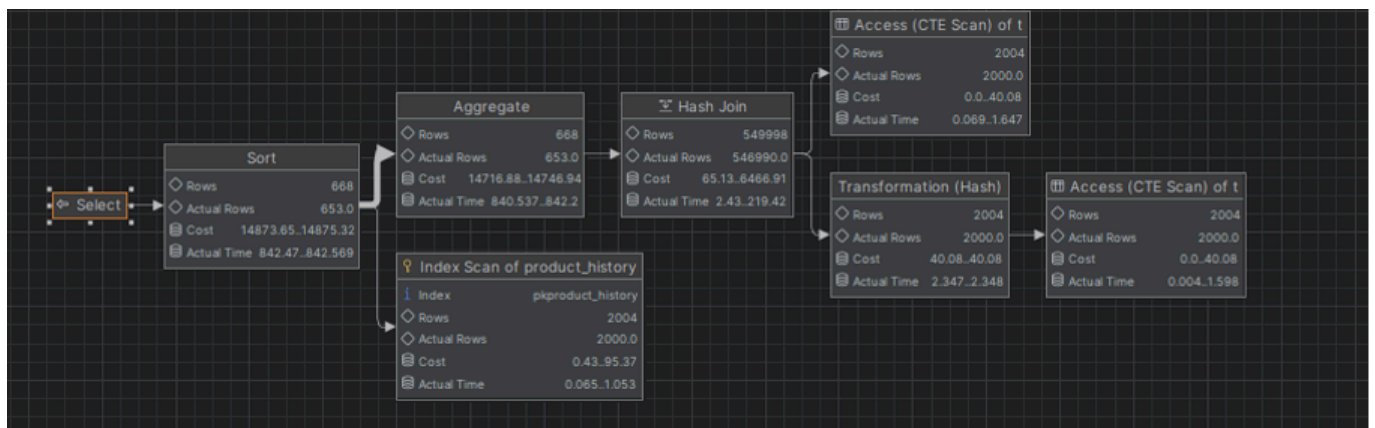


Wyniki zapytań dla PostgreSQL:

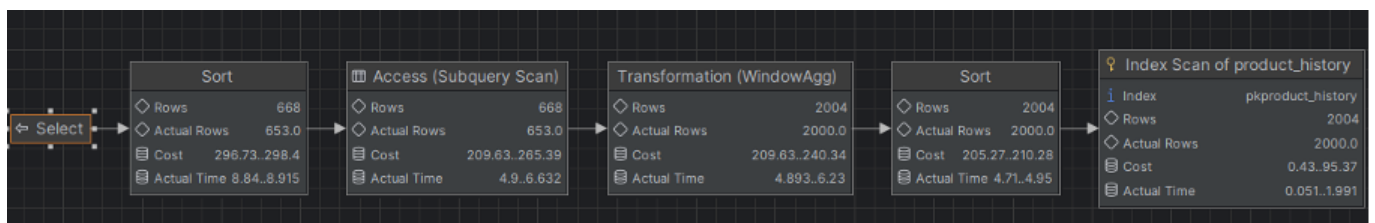
- podzapytanie



- join

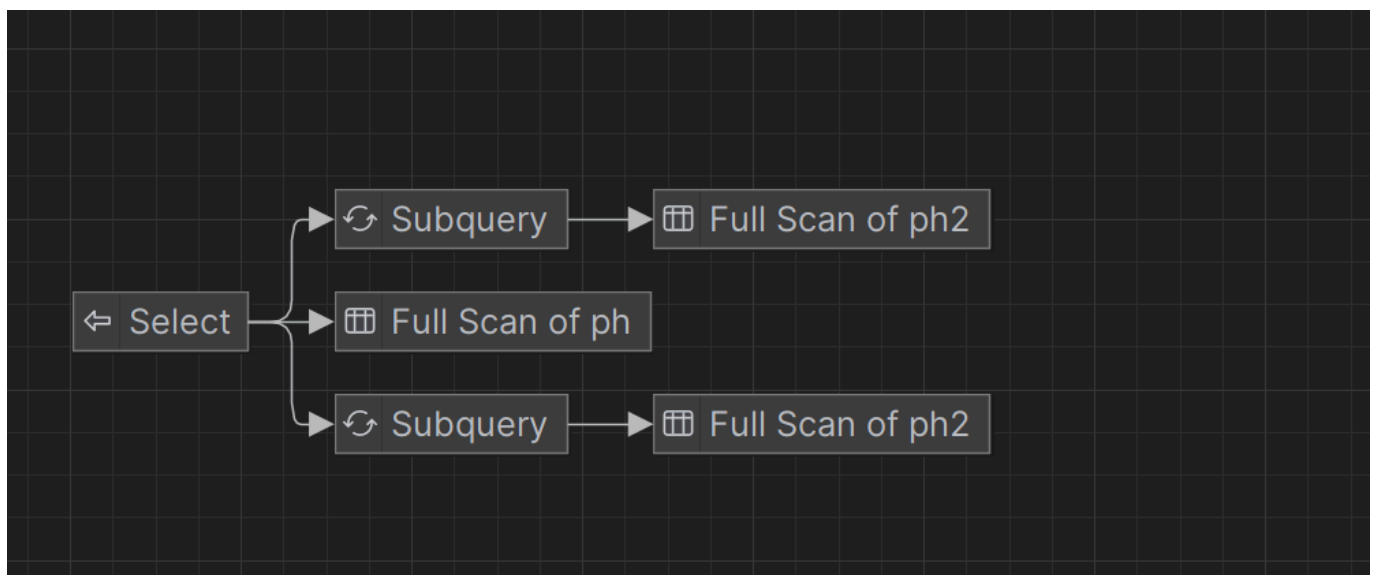


- funkcja okna

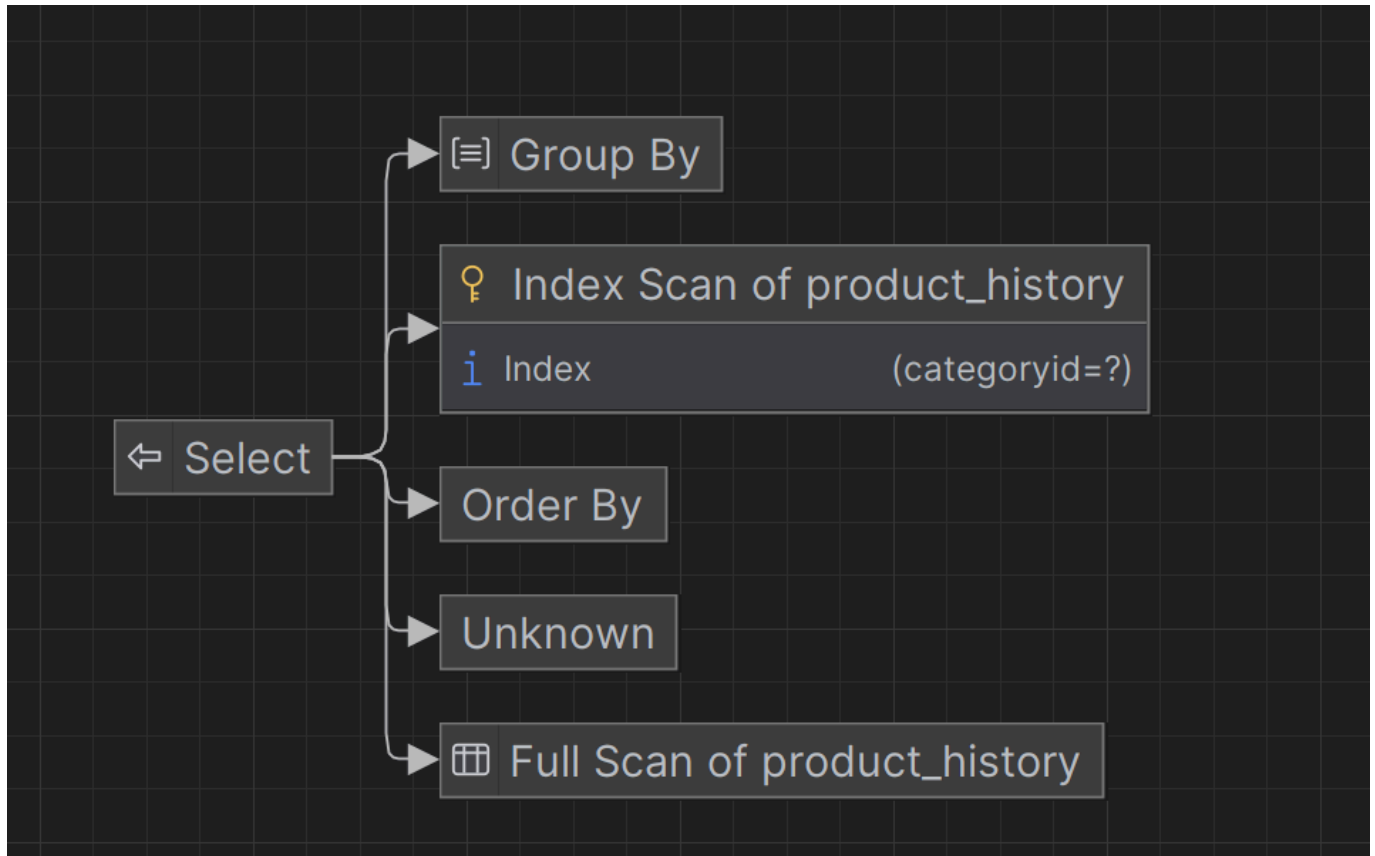


Wyniki zapytań dla SQLite:

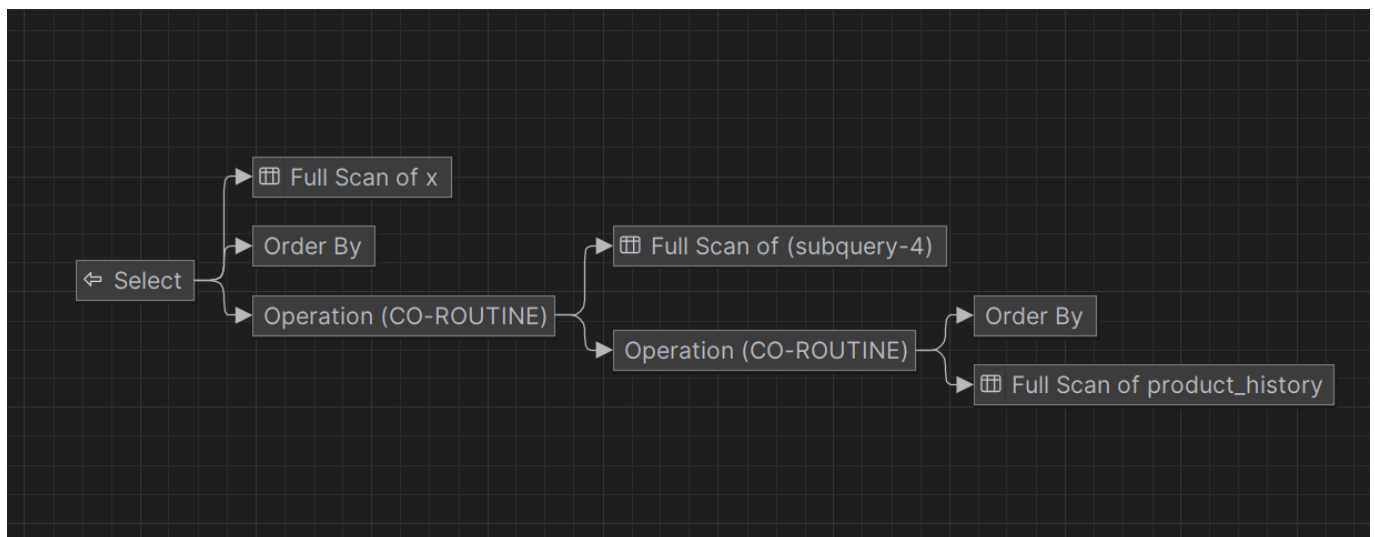
- podzapytanie



- join



- funkcja okna



Wnioski

MS SQL Server

- Podzapytanie ma Clustered Index Seek i Clustered Index Scan
- Join ma dwa Clustered Index Seek
- Funkcja okna ma Clustered Index Seek i Sort, ma najwięcej operacji

PostgreSQL

- Join ma koszt 14875.32, używa Hash Join i Sort
- Funkcja okna ma koszt 298.4, używa Index Scan, Sort i WindowAgg, ma najlepszy koszt
- Podzapytanie ma koszt 88388.85, używa Full Scan dwa razy, ma najwyższy koszt

SQLite

- Podzapytanie wykonuje trzy Full Scan tabeli product_history
- Join używa Index Scan po categoryid i Full Scan
- Funkcja okna ma najwięcej operacji

zadanie	pkt
1	1
2	1
3	1
4	1
5	1
6	2
razem:	7